Especificación de Requisitos del Software (ERS)

Tienda online Level-Up Gamer — Versión 2.0 (frontend con React)

Asignatura: DSY1104 Desarrollo Fullstack II · Evaluación Parcial 2 · Duoc UC

Integrantes: Matias Korner, Patricicio Segovia, Leonardo Bravo

Control de versiones

Versión	Etapa	Descripción
1.0	Evaluación Parcial 1	Tienda en HTML, CSS, JavaScript.
2.0	Evaluación Parcial 2	Migración a React, componentes reutilizables, archivo de datos con CRUD, nuevas vistas y pruebas unitarias

1. Introducción

Este documento especifica los requisitos de la versión 2.0 de **Level-Up Gamer**, una tienda online para gamers en Chile, desarrollada como frontend con React. Cubre la tienda pública, una vista de administración inicial, una fuente de datos simulada con CRUD y las pruebas unitarias. Quedan fuera el backend real y los pagos reales, y todos los requisitos están sujetos a validación del docente.

La versión 1.0 (Evaluación 1) fue la tienda en HTML, CSS y JavaScript. La versión 2.0 (Evaluación 2) agrega la migración a React, los componentes, los datos con CRUD y las pruebas unitarias.

1.2 Definiciones y siglas

Término	Significado
ERS	Especificación de Requisitos del Software
SPA	Aplicación de página única (Single Page Application)
Componente	Pieza reutilizable de interfaz en React, escrita como función
Props	Datos y funciones que un componente recibe desde su componente
Estado	Memoria interna de un componente (`useState`) para valores que cambian
CRUD	Crear, Leer, Actualizar y Eliminar
Vitest	Herramienta de pruebas unitarias y de integración rápida para aplicaciones basadas en Vite y React.
Puntos LevelUp	Puntos del programa de fidelización de la tienda

2. Descripción general

La aplicación es una SPA (aplicación de página única) hecha con React y Vite, con navegación mediante React y diseño responsivo con Bootstrap 5. Los datos se guardan en `localStorage` a través de (productos con CRUD), lo que simula una base de datos.

2.2 Funciones principales

- Registro e inicio de sesión de usuarios, con beneficio de descuento para correos Duoc.
- Exploración del catálogo con categorías, búsqueda, filtros y orden.
- Detalle de producto y carrito de compras con resumen de totales.
- Proceso de pago con datos del cliente y dirección de entrega, y confirmación de la compra.
- Perfil de usuario, historial de pedidos, referidos y puntos LevelUp.
- Blog y soporte técnico por WhatsApp.
- Vista de administración.

2.3 Tipos de usuario

Usuario	Descripción
Visitante	Navega el catálogo, el blog y el soporte sin iniciar sesión.
Cliente registrado	Compra, gestiona su perfil, ve sus pedidos y acumula puntos LevelUp.
Cliente Duoc	Cliente registrado con correo @duoc.cl o @profesor.duoc.cl, con 20% de descuento.
Administrador	Accede al panel de administración.

3. Requisitos funcionales

ID	Requisito	Estado	Módulo
RF-01	Registro solo para mayores de 18 años, con código de referido.	Implementado	RegistroUsuario
RF-02	Descuento de 20% para correos @duoc.cl y @profesor.duoc.cl.	Implementado	services/tienda.js
RF-03	Inicio de sesión y perfil de usuario.	Implementado	Login, Perfil
RF-04	Catálogo por categorías con búsqueda, filtros y orden.	Implementado	Catalogo, TarjetaProducto
RF-05	Detalle de producto.	Implementado	DetalleProducto

ID	Requisito	Estado	Módulo
RF-06	Carrito: agregar, cambiar cantidad, eliminar y ver totales.	Implementado	Carrito, ItemCarrito
RF-07	Pago con datos del cliente y dirección (región y comuna) validados.	Implementado	ProcesoPago
RF-08	Confirmación de compra e historial de pedidos.	Implementado	ConfirmacionPedido, ListaPedidos
RF-09	Puntos LevelUp, niveles y canje.	Implementado	SistemaPuntos
RF-10	Blog y soporte técnico por WhatsApp.	Implementado	Blog, Soporte
RF-11	Fuente de datos de productos con CRUD.	Parcial	data/productos.js
RF-12	Panel de administración.	Parcial	MenuAdmin
RF-13	Vistas de ofertas, categorías y pago con error.	Pendiente	—
RF-14	Reseñas, mapa de eventos y recomendaciones.	Pendiente	—

4. Requisitos no funcionales

- **Responsividad:** diseño adaptable a móviles, tabletas y escritorios con Bootstrap 5.
- **Identidad visual:** fondo negro, acentos azul #1E90FF y verde #39FF14, y tipografías Roboto y Orbitron, según el caso.
- **Mantenibilidad:** cada componente tiene una sola responsabilidad, recibe datos por props y usa estado solo cuando es necesario.
- **Testabilidad:** las pruebas se repiten con `npm test`.
- **Compatibilidad:** navegadores modernos; las pruebas se ejecutan en un navegador simulado (jsdom).
- **Seguridad:** `localStorage` es solo de demostración y no es seguro para producción.

5.0 Componentes de la aplicación

La aplicación tiene 24 componentes de React: `App`, 3 de estructura (`Layout`, `Header` y `Footer`), 4 transversales o reutilizables (`ScrollToTop`, `ErrorBoundary`, `TarjetaProducto` e `ItemCarrito`) y 16 páginas, una por cada vista. Todos son funciones, salvo `ErrorBoundary`, que es una clase porque React solo permite capturar errores de renderizado con una. Además hay 2 hooks personalizados que dan apoyo a los componentes.

5.1 Componentes de estructura y transversales

Componente	Qué hace	Props	Estado y efectos
App (src/App.jsx)	Define las rutas con Routes y Route. Agrupa 8 vistas dentro de Layout; las demás quedan fuera para no mostrar la barra de navegación.	Ninguna	useLocation: entrega la key de la ruta a ErrorBoundary.
Layout (components/layout)	Arma la estructura común: Header, el contenido de la vista (Outlet) y Footer.	Ninguna	Sin estado.
Header (components/layout)	Barra de navegación Bootstrap con logo, enlaces (Inicio, Catálogo, Blog, Soporte), carrito con contador y ícono de perfil. Sin sesión muestra Iniciar sesión y Registrarse, y un modal para invitados. En móvil el menú se pliega.	Ninguna	Estado cantidadProductos y hook useSesion. Efectos: actualiza el contador con los eventos storage, focus, lug-carrito-actualizado y lug-cart-updated; cierra el menú móvil al cambiar de ruta; limpia el fondo del modal al desmontarse.
Footer (components/layout)	Pie de página con 4 columnas: logo y descripción, navegación, soporte y contacto.	Ninguna	Sin estado.
ScrollToTop (components)	Sube al inicio de la página cada vez que cambia la ruta o el parámetro de búsqueda. No dibuja nada (devuelve null).	Ninguna	useEffect con pathname y search.
ErrorBoundary (components)	Captura errores de renderizado y muestra PaginaError con código 500. Al navegar a otra ruta vuelve a intentar el renderizado normal.	children, resetKey	Estado hayError; métodos getDerivedStateFromError, componentDidCatch y componentDidUpdate.

5.1.2 Componentes reutilizables

Son componentes de presentación: no guardan datos, todo les llega por props.

Componente	Qué muestra	Props	Usado en
TarjetaProducto	Un producto del catálogo: imagen desde /img, nombre, precio con formato es-CL y enlace «Ver Detalle» a /detalle?id=.	producto (id, nombre, precio, imagen, alt, categorias)	Catalogo

Componente	Qué muestra	Props	Usado en
`ItemCarrito`	Un producto del carrito: imagen, nombre, variante, precio por cantidad, botón de eliminar y botones «-» y «+».	`item`, `onCambiarCantidad(id, cambio)`, `onEliminar(id)`	`Carrito`
`PaginaError`	Pantalla de error con código, subtítulo y descripción, y un modal para reportar el problema con validación del formulario.	`codigo` (por defecto 404), `subtitulo`, `descripcion`	Rutas `/404` y `*`, y `ErrorBoundary` (500)

5.1.3 Páginas (una por vista)

Página	Ruta	Qué hace	Estado y datos
`Inicio`	`/`	Banner CYBER STORM RECARGADO y artículos destacados filtrables por categoría.	Estado `categoriaActual`. La lista de productos está definida en el mismo archivo.
`Catalogo`	`/catalogo`	Catálogo con categorías principales y menú «más», búsqueda por texto y orden (precio, nombre o categoría). Dibuja una `TarjetaProducto` por producto.	Estados `categoriaActual`, `textoBusqueda`, `criterioOrden`, `menuAbierto` y `categoriaActiva`. Productos desde `obtenerProductos()`.
`DetalleProducto`	`/detalle`	Detalle del producto indicado en `?id=`, con selector de cantidad y botón para agregar al carrito. Si el id no existe muestra «Producto no encontrado».	Estado `cantidad`; `useSearchParams` y `useNavigate`. Productos definidos en el mismo archivo; `LevelUp.Carrito.agregar`.
`Carrito`	`/carrito`	Lista los productos con `ItemCarrito`, cambia cantidades, elimina y muestra subtotal, descuento y total. Muestra un mensaje si el carrito está vacío.	Estado `items`; `LevelUp.Carrito`.
`Perfil`	`/perfil`	Datos del usuario, nivel y puntos; formulario con validación para editar nombre, correo y teléfono; opciones «próximamente» y cierre de sesión. Sin sesión redirige a `/login`.	Estados `usuario`, `formulario`, `errores` e `intentado`; `LevelUp.Usuario` y `LevelUp.Validar`.

Página	Ruta	Qué hace	Estado y datos
Blog	/blog	Artículo destacado y lista de artículos filtrable por categoría.	Estado categoriaActual; data/articulosBlog.js; hook useBodyClass.
ArticuloN1	/articuloN1	Muestra el artículo indicado en ?id=. Si no existe, ofrece volver al blog.	useSearchParams; data/articulosBlog.js; hook useBodyClass.
Soporte	/soporte	Preguntas frecuentes en acordeón (4 preguntas) y tarjeta de soporte por WhatsApp.	Preguntas definidas en el mismo archivo; hook useBodyClass.
ProcesoPago	/pago	Pago en 3 pasos (Envío, Pago y Confirmar) con datos y dirección (región), método de envío, método de pago y tarjeta, todo validado. Al confirmar crea el pedido, suma puntos si hay sesión y vacía el carrito. Con el carrito vacío redirige a /carrito.	Estados paso, envio, errores, intentadoEnvio, metodoEnvio, metodoPago y tarjeta; LevelUp.Carrito, Pedidos y Usuario.
ConfirmacionPedido	/confirmacion	Resumen del pedido confirmado o en seguimiento: fecha, total y puntos ganados. Sin pedidos redirige a /pedidos.	Estado pedido; LevelUp.Pedidos.
ListaPedidos	/pedidos	Historial con filtros (Todos, En camino, Entregado y Cancelado). «Rastrear» abre la confirmación y «Repetir pedido» agrega los productos de nuevo al carrito. Sin sesión redirige a /login.	Estados pedidos y filtroActivo; LevelUp.Pedidos.
SistemaPuntos	/puntos	Nivel, barra de progreso, recompensas canjeables y logros. Al canjear descuenta puntos y avisa si no alcanzan. Sin sesión redirige a /login.	Estado usuario; LevelUp.Usuario.
Login	/login	Acceso de cliente y de administración (se alterna con modoAdmin), con validación de correo y contraseña y un error por campo. Permite entrar como invitado. El acceso de administración usa correos @levelup.cl, guarda el rol y lleva a /admin.	Estados modoAdmin, correo, contrasena y sus errores (cliente y administración); hook useBodyClass; localStorage.

Página	Ruta	Qué hace	Estado y datos
`RegistroUsuario`	`/registro`	Formulario de registro con nombre, correo (`@duoc.cl`, `@profesor.duoc.cl` o `@gmail.com`), contraseña de 4 a 10 caracteres, términos y código de referido. Guarda el usuario y muestra una alerta (SweetAlert2) antes de ir a `/login`.	Estado `datos` (un solo objeto para todos los campos); `useNavigate`.
`MenuAdmin`	`/admin`	Panel con secciones Inicio, Productos, Pedidos y Usuarios según el rol (Administrador o Vendedor). El administrador puede agregar, editar y eliminar productos. Sin rol redirige a `/login`.	Estados `seccionActiva` y `productos`; `localStorage` (`lug_rol`, `lug_orders`, `lug_usuario_registrado`) .
`PaginaError`	`/404` y `*`	Pantalla de error con modal de reporte (ver 5.1.3).	Estado `validado`; referencias a los modales.

5.1.5 Hooks personalizados

Hook	Qué hace	Usado en
`useSesion()`	Devuelve `true` si hay una sesión iniciada (clave `lug_user` en `localStorage`). Se actualiza al cambiar de ruta y con los eventos `storage` y `lug-sesion-actualizada`.	`Header`
`useBodyClass(clase)`	Agrega una clase al `<body>` mientras el componente está montado y la quita al salir.	`Login`, `Blog`, `ArticuloN1` y `Soporte`

6.0 Gestión de props y estado

- **Los datos bajan por props:** `Carrito` guarda la lista de productos en su estado y entrega cada uno a un `ItemCarrito` (`item`), junto con las funciones `cambiarCantidad` y `eliminarProducto`.
- **Los eventos suben por funciones:** al pulsar «+», `ItemCarrito` llama a `onCambiarCantidad(id, 1)`. `Carrito` actualiza `localStorage` y vuelve a leer la lista, y la pantalla se actualiza.
- **Catálogo:** `Catalogo` mantiene en estado la categoría, el texto de búsqueda y el criterio de orden, y entrega cada producto a una `TarjetaProducto`.
- **Formularios:** `Login` guarda en estado el correo, la contraseña y el mensaje de error de cada campo.

6.2 Fuente de datos de productos

`data/productos.js` actúa como base de datos simulada: la primera vez guarda los 13 productos iniciales en `localStorage` (clave `lug_productos`) y el catálogo lee desde ahí.

Función	Qué hace	Devuelve
`obtenerProductos()`	Lee todos los productos.	Arreglo de productos
`obtenerProductoPorId(id)`	Busca un producto por su id.	El producto, o `undefined`
`crearProducto(datos)`	Agrega un producto nuevo.	El producto creado, o `null` si falta el id o ya existe
`actualizarProducto(id, cambios)`	Cambia los campos indicados; el id no se puede modificar.	El producto actualizado, o `null` si no existe
`eliminarProducto(id)`	Borra un producto.	`true` si lo borró, `false` si no existía
`restablecerProductos()`	Vuelve a los datos iniciales.	Arreglo de productos

7. Pruebas unitarias

El proyecto tiene **36 pruebas unitarias en 5 archivos**, escritas con Vitest y Testing Library, que verifican componentes, páginas y la lógica de datos.

7.1 Herramientas y configuración

Herramienta	Uso
Vitest	Escribe y ejecuta las pruebas (`describe`, `it`, `expect`, `beforeEach`, `vi.fn`). Se lanza con `npm test`, que ejecuta `vitest run`.
jsdom	Simula el navegador (DOM, formularios y `localStorage`) para que las pruebas corran sin abrir uno.
Testing Library	Dibuja los componentes (`render`), busca elementos (`screen`, `within`) y simula eventos (`fireEvent`).
`@vitest/coverage-v8`	Mide la cobertura de código.
Configuración	Está en `vite.config.js`: entorno `jsdom`, variables globales activadas y funciones simuladas reiniciadas entre pruebas (`clearMocks`, `restoreMocks`).

7.2 Tipos de prueba aplicados

Tipo	Qué verifica	Ejemplo
Renderizado	Que el componente muestre los datos recibidos.	`ItemCarrito`: nombre, variante y cantidad
Renderizado condicional	Que se muestren u oculten elementos según el caso.	`Carrito` vacío frente a `Carrito` con productos
Props	Que se usen las propiedades recibidas.	`TarjetaProducto`: nombre, precio, imagen y enlace
Eventos	Que un clic ejecute la acción esperada.	`ItemCarrito` llama a `onCambiarCantidad(id, 1)`
Estado y persistencia	Que el cambio se vea en pantalla y se guarde en `localStorage`.	`Carrito`: sumar una unidad y recalcular el total
Validación de formularios	Que se muestren los errores y se bloquee el envío.	`Login`: contraseña incorrecta y campos vacíos
Lógica de datos	Que el CRUD cumpla sus reglas.	`productos.js`: crear, actualizar y eliminar
Navegación	Que un enlace lleve a la vista correcta.	`TarjetaProducto`: clic en «Ver Detalle»

7.3 Casos de prueba

Archivo	Caso	Tipo
`ItemCarrito.spec.jsx` (6 casos)	Muestra nombre, variante y cantidad recibidos por props	Props
	Muestra el precio multiplicado por la cantidad	Props
	Agrega «/» al inicio de la ruta de la imagen si no lo trae	Props
	Clic en «+» llama a `onCambiarCantidad` con (id, 1)	Evento
	Clic en «-» llama a `onCambiarCantidad` con (id, -1)	Evento
	Clic en el tacho llama a `onEliminar` con el id	Evento
`TarjetaProducto.spec.jsx` (7 casos)	Muestra el nombre del producto	Renderizado
	Muestra el precio con el formato de la tienda	Props

Archivo	Caso	Tipo
	Muestra la imagen desde `/img` con su texto alternativo	Props
	Guarda las categorías en `data-categoria`	Props
	El enlace apunta a `/detalle` con el id del producto	Props
	Codifica el id si trae caracteres especiales	Props
	Clic en «Ver Detalle» navega a la pantalla de detalle	Navegación
`Carrito.spec.jsx` (7 casos)	Muestra todos los productos guardados en el carrito	Renderizado
	Muestra la cantidad total de unidades en el subtítulo	Renderizado
	Con el carrito vacío muestra el mensaje y oculta el resumen	Condicional
	Con productos oculta el mensaje y muestra el botón de pago	Condicional
	Clic en «+» aumenta la cantidad y actualiza el total	Estado
	Clic en «-» con cantidad 1 elimina el producto	Estado
	Clic en el tacho elimina el producto y deja el carrito vacío	Estado
`Login.spec.jsx` (2 casos)	Con contraseña incorrecta muestra el error y no inicia sesión	Validación
	Con los campos vacíos bloquea el envío del formulario	Validación
`productos.spec.js` (14 casos)	Lectura (5): devuelve los 13 iniciales, los guarda, busca por id, devuelve `undefined` si no existe y se recupera de un JSON dañado	Datos
	Creación (3): agrega y guarda, rechaza un id repetido y rechaza un producto sin id	Datos
	Actualización (3): cambia solo los campos indicados, no deja cambiar el id y devuelve `null` si no existe	Datos

Archivo	Caso	Tipo
	Eliminación (2): borra y devuelve `true`, o devuelve `false` si no existe	Datos
	Restablecer (1): vuelve a los datos iniciales	Datos

7.4 Mocks y datos de prueba

- **Funciones simuladas (vi.fn):** reemplazan las funciones que el componente recibe del padre, para comprobar que se llaman con los argumentos correctos (`ItemCarrito`) o que un evento no se dispara (`Login`).
- **MemoryRouter y Routes:** simulan el router de la aplicación en los componentes con enlaces o redirecciones (`TarjetaProducto`, `Carrito`, `Login`).
- **localStorage de prueba:** se limpia antes y después de cada prueba y se llena con productos de ejemplo, así las pruebas no dependen unas de otras.